

```

\document class[a4paper, 12pt]{article}

\use package {amsmath,amssymb}

\use package {modern}

\usepackage{hyperref}

\usepackage{tcolorbox} % Package to create framed boxes

\usepackage{geometry}


% Set the margins to create a more portrait-like layout
\geometry{
    top=2cm,
    bottom=2cm,
    left=2.5cm,
    right=2.5cm,
}


\title{LUDO-CS Assignment Report}

\author{University of Colombo School of Computing\\Author:
S.K.D.Sanjula\\Index Number: 23001763\\Date: 01/09/2024}

\date{}


\begin{document}


\maketitle


\tableofcontents

\newpage


\section{Introduction}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Introduction]
{
Ludo-CS is an extended version of the traditional Ludo game, \\

```

incorporating additional rules, strategies, and behaviors \\ that add complexity and competitiveness to the gameplay. \\ This report provides an in-depth analysis of the program's \\ structure, memory allocation methods, and efficiency. \\ The program is implemented in the C programming language, \\ which allows for detailed control over memory management \\ and optimization.

}

\section{Structures Used to Represent the Board and Pieces}

\subsection{Piece Position Tracking}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Piece Position Tracking]

{

The core of the game's state is maintained through various \\ arrays that track the position, status, and ownership \\ of pieces on the board:

\textbf{Piece Positions Array:} The array \texttt{piece_positions} \\ is used to track the location of each piece on the board. \\ This array is crucial for determining each piece's progress \\ towards reaching home. A value of -1 indicates that the \\ piece is still in the base, while any other non-negative \\ integer represents its position on the board or in the \\ home stretch.

\textbf{Piece in Home Stretch Array:} The \\ \texttt{piece_in_home_stretch} \\ array indicates whether a piece has entered the home stretch, \\ where the movement rules differ and exact rolls are required \\

to reach the home cell.

}

\subsection{Block Logic}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Block Logic]

{

Blocks are a strategic feature in Ludo-CS that allow players \\
to control key positions on the board:

\textbf{Block Size and Ownership:} The \texttt{block_size} array \\
tracks the number of pieces in a block at each position, while \\
the \texttt{block_owner} array identifies which player owns \\
the block. When multiple pieces of the same player occupy the \\
same position, a block is formed, preventing opponents from \\
passing through the blocked cell.

}

\subsection{Special Effects}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Special Effects]

{

Ludo-CS introduces various special effects that affect gameplay:

\textbf{Teleportation and Status Effects:} The game uses arrays \\
like \texttt{energized}, \texttt{sick}, and \texttt{briefing} to \\
manage status effects applied to pieces. These arrays track \\
the duration and intensity of these effects, ensuring that the \\
game state reflects the current status of all pieces.

}

\subsection{Mystery Box Mechanism}

```
\tcolorbox[colback=yellow!10, colframe=blue!50, title=Mystery Box  
Mechanism]
```

```
{
```

The mystery box introduces randomness into the game:

```
\textbf{Mystery Box State:} The \texttt{current\_mystery\_box\_cell}  
\
```

variable tracks the location of the active mystery box, and \\
the \texttt{mystery_round_counter} variable monitors how \\
long the mystery box will remain active. This mechanism \\
adds an element of surprise, forcing players to adapt \\
their strategies as the game progresses.

```
}
```

```
\section{Function Implementations}
```

```
\subsection{initialize\_game() }
```

```
\tcolorbox[colback=yellow!10, colframe=blue!50,  
title=initialize\_game() ]
```

```
{
```

The \texttt{initialize_game()} function is responsible for setting up \\
\

the initial game state. It initializes all arrays to their default \\
values, ensuring that the game starts with all pieces in the base, \\
no active blocks, and no ongoing special effects.

```
}
```

```
\subsection{play\_game() }
```

```
\tcolorbox[colback=yellow!10, colframe=blue!50, title=play\_game() ]
```

```
{
```

The \texttt{play_game()} function is the heart of the Ludo-CS
program, \\
\

managing the main game loop. It handles the sequence of player turns,
 \\

including dice rolls, piece movements, and the application of special
 \\

effects. The function also checks for the presence of a winner at the
 \\

end of each round.

}

\subsection{roll_dice()}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=roll_dice()]

{

The \texttt{roll_dice()} function simulates the roll of a six-sided
 die, \\

returning a random number between 1 and 6. This function is essential
 \\

for determining the movement options available to players during their
 turn.

}

\subsection{coin_toss()}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=coin_toss()]

{

The \texttt{coin_toss()} function determines the direction in which
 \\

a piece will move—either clockwise or counterclockwise. This decision
 \\

adds an element of randomness to the game, requiring players to adapt
 \\

their strategies based on the direction determined by the toss.

}

\subsection{move_piece()}

```
\tcolorbox[colback=yellow!10, colframe=blue!50, title=move\_piece()]
{
The \texttt{move\_piece()} function updates the position of a piece \\
on the board based on the dice roll and the current game state. \\
It accounts for the player's direction, any special effects \\
(such as teleportation or status effects), and block logic.
}
```

```
\subsection{find\_piece\_to\_move()}
\tcolorbox[colback=yellow!10, colframe=blue!50,
title=find\_piece\_to\_move()]
{
The \texttt{find\_piece\_to\_move()} function selects which piece \\
a player should move based on the dice roll and the current positions \\
of all pieces. This function prioritizes pieces that are already \\
on the board and can make progress towards the home.
}
```

```
\subsection{capture\_piece()}
\tcolorbox[colback=yellow!10, colframe=blue!50,
title=capture\_piece()]
{
The \texttt{capture\_piece()} function manages the capture of opponent \\
pieces. When a player's piece lands on a square occupied by an \\
opponent's piece, the opponent's piece is sent back to its base. \\
This function adds a competitive element to Ludo-CS, as players \\
can target opponent pieces to hinder their progress.
}
```

```
\subsection{handle\_block\_logic() }
```

```
\tcolorbox[colback=yellow!10, colframe=blue!50,
title=handle\_block\_logic()]

{

The \texttt{handle\_block\_logic()} function manages the creation \
and dissolution of blocks on the board. When multiple pieces of \
the same player occupy the same position, a block is formed, \
preventing opponents from passing through.

}
```

```
\subsection{print\_piece\_locations()}

\tcolorbox[colback=yellow!10, colframe=blue!50,
title=print\_piece\_locations()]

{

The \texttt{print\_piece\_locations()} function provides a visual \
representation of the current positions of all pieces on the board.

}
```

```
\subsection{check\_winner()}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=check\_winner()]

{

The \texttt{check\_winner()} function determines if any player \
has moved all their pieces to the home, thereby winning the game.

}
```

```
\subsection{spawn\_mystery\_box()}

\tcolorbox[colback=yellow!10, colframe=blue!50,
title=spawn\_mystery\_box()]

{

The \texttt{spawn\_mystery\_box()} function introduces an element \
of unpredictability by randomly placing a mystery box on the board. \
This box can trigger various effects when a piece lands on it, \
```

such as teleportation or status changes.

}

\subsection{Special Teleportation Effects}

\toolbox[colback=yellow!10, colframe=blue!50, title=Special Teleportation Effects]

{

Special teleportation effects, managed by functions like

\texttt{teleported_to_bhawana()}, \\\

\texttt{teleported_to_kotuwa()}, and

\texttt{teleported_to_pitakotuwa()}, \\\

add complexity to Ludo-CS.

}

\section{Justification for the Used Structures}

\subsection{Arrays for State Management}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Arrays for State Management]

{

Arrays were chosen as the primary data structure for managing \\\

the game state due to their simplicity, fixed memory allocation, \\\

and constant-time access operations.

}

\subsection{Memory Considerations}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Memory Considerations]

{

By using static memory allocation, the program ensures that \\\

all necessary memory is reserved at compile time, reducing the \\\

risk of runtime errors and improving the predictability of \\\

memory usage. This approach simplifies memory management, as \\ there is no need for dynamic allocation or deallocation during \\ the game.

}

\section{Memory Allocation Considerations}

\subsection{Static Memory Allocation}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Static Memory Allocation]

{

Static memory allocation is used for most arrays in the Ludo-CS program. \\

This type of allocation occurs at compile time, meaning that the size \\

and location of each array are fixed before the program runs.

\textbf{Advantages:} \\

- \textbf{Simplicity:} Managing memory is simpler with static allocation, \\

as there is no need to allocate or deallocate memory during runtime. \\

- \textbf{Efficiency:} Static allocation allows for fast access to memory, \\

as the addresses of all arrays are known at compile time.

}

\subsection{Dynamic Memory Allocation}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Dynamic Memory Allocation]

{

While the current implementation of Ludo-CS relies on static memory \\

allocation, dynamic memory allocation could be used to provide greater
\\

flexibility. Dynamic memory allocation allows the program to allocate
\\

memory at runtime based on the actual requirements of the game.

\textbf{Advantages:} \\

- \textbf{Flexibility:} Memory is allocated as needed, making it more
\\

adaptable to different scenarios. \\

- \textbf{Efficient Memory Usage:} Memory is allocated only when
needed, \\

potentially reducing wastage.

}

\section{Discussion on Program Efficiency}

\subsection{Time Complexity}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Time Complexity]

{

The time complexity of the Ludo-CS program is determined by the
operations \\

performed during each player's turn and the total number of rounds
until \\

the game concludes.

\textbf{roll_dice Function:} The time complexity of rolling the dice
is $O(1)$, \\

as it involves a simple random number generation and a modulo
operation.

\textbf{find_piece_to_move Function:} This function operates in
constant time, $O(1)$, \\

as it checks each piece's position and status to determine which piece should move.

\textbf{Overall Time Complexity:} the overall time complexity of the Ludo-CS \\\

program is $O(R * P)$, where R is the number of rounds and P is the number of players.

}

\subsection{Space Complexity}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Space Complexity]

{

The space complexity of the Ludo-CS program is primarily $O(P * N)$, \\\ reflecting the use of arrays to store the positions, statuses, and \\\ properties of each piece. The space requirements are determined by the \\\

Number of players (P) and the number of pieces per player (N).

}

\section{Conclusion}

\tcolorbox[colback=yellow!10, colframe=blue!50, title=Conclusion]

{

The Ludo-CS program is a robust simulation of a more complex version \\\

of the traditional Ludo game. By leveraging static memory allocation, \\\

the program is efficient and straightforward to manage, with minimal \\\

overhead. The program's design ensures that it operates within \\\ acceptable time and space constraints for typical gameplay scenarios.

}

```
\end{document}
```